Tutorial Series No. 02

Dr. Rochdi Boudjehem

2025-11-06

Chapter 2: Paradigms for Parallel Applications - Guided Tutorial

This tutorial builds directly on Chapter 2 (memory architectures, parallel application paradigms, granularity, execution models: MPI, OpenMP, PThreads). It blends conceptual classification, design exercises, practical MPI/OpenMP/PThreads coding problems, and short reasoning tasks.

Part 1 - Paradigms for Parallel Applications

Exercise 2.1: Identify the Memory Model (Conceptual)

For each scenario, classify the primary memory architecture as one of: Distributed Memory, Shared Memory, Accelerated (CPU+GPU), or Hybrid. Justify briefly (2–3 lines).

1. A weather simulation runs across 64 nodes; each node has 2 × 24-core CPUs and an Infiniband interconnect. Processes exchange halo regions every time step.
2. A desktop application processes a 4K image using 12 CPU threads sharing a single address space.
3. Training a CNN on a single workstation using a 12-core CPU and an RTX GPU; data batches are copied to the device, then processed by kernels.
4. A multi-node CFD solver uses MPI between nodes and OpenMP threads inside each process.

Exercise 2.2: Programming Models ↔ Architectures (Mapping)

Fill in the most natural pairing (primary intent). You may list secondary uses if reasonable.

- OpenMP → _____
- Pthreads → _____
- MPI → _____
- CUDA/OpenCL → _____
- MPI + OpenMP → _____

Exercise 2.3: Concurrency vs Parallelism (Distinguish)

For the situations below, mark C (concurrent), P (parallel), or Both (explain why). 1–2 lines each.

1. An OS time-slices 200 runnable tasks on a dual-core laptop.
2. A pipeline with 5 stages (fetch, decode, ...) executing on different hardware units per clock.
3. A web server cluster with multiple machines handling independent requests.
4. A single-core app using async I/O and callbacks to keep the UI responsive during file loads.

Exercise 2.4: Decomposition Choice

Choose Task Parallelism, Data Parallelism, or a Hybrid. Justify the choice and outline 3–5 bullet steps for the decomposition.

1. Apply a Gaussian blur to a large image (7680×4320), outputting a new image.
2. ETL pipeline: ingest CSVs → clean/normalize → transform → persist to DB.
3. Compute N independent Monte Carlo simulations for risk estimation.
4. Video encoding service offering H.264, HEVC, and AV1 outputs from the same source.

Exercise 2.5: Granularity Leveling

Label the following as fine-, medium-, or coarse-grained. Describe the main trade-off (latency vs utilization) in 1–2 lines.

1. Assign one thread per pixel for a simple per-pixel transform on CPU.
2. Assign one process per 2D tile (256×256) for image convolution across a cluster.
3. Run 200 independent parameter sweeps, each taking 20 minutes, one per node.

Part 2 - MPI Fundamentals and Programming

Exercise 2.6: MPI Trapezoidal Rule (Coding Problem)

Implement parallel numerical integration using the trapezoidal rule in MPI. The function to integrate is $f(x) = x^2$ from a=0 to b=1.

Requirements: - Use SPMD model with rank-based work distribution - Process 0 (root) should collect results and print the final integral - Each process computes its local trapezoidal approximation - Use MPI_Send/MPI_Recv for communication

Skeleton Code:

```c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

double f(double x) {
    return x * x;  // f(x) = x²
}

double trapezoidal_rule(double local_a, double local_b, int local_n, double h)
{
    // Implement local trapezoidal rule computation
    // Return local integral approximation
}

int main(int argc, char** argv) {
    // MPI initialization
    // Get rank and size
    // Parse command line for n (number of trapezoids)
    // Compute local a, b for this rank
    // Compute local integral
    // Send to root or receive from workers
    // Root prints final result
    // MPI finalization
}
```

Test with: `mpirun -np 4 ./trapezoidal 1000`

Exercise 2.7: MPI Point-to-Point Communication Patterns

For each scenario, choose the appropriate MPI communication pattern and justify:

1. Root process broadcasts initial data to all workers.
2. Workers send computed results back to root for aggregation.
3. Two neighboring processes exchange boundary data in a 1D decomposition.
4. All processes need the same final result (reduction).

Exercise 2.8: MPI Collective Operations

Convert the following point-to-point communications to collectives where appropriate:

```
// Original code - root broadcasts to all
if (rank == 0) {
    for(int i=1; i<size; i++) {
        MPI_Send(&data, 1, MPI_DOUBLE, i, 0, MPI_COMM_WORLD);
    }
} else {
    MPI_Recv(&data, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD, &status);
}

// Original code - workers send results to root
if (rank != 0) {
    MPI_Send(&local_sum, 1, MPI_DOUBLE, 0, 1, MPI_COMM_WORLD);
} else {
    double total = local_sum;
    for(int i=1; i<size; i++) {
        double temp;
        MPI_Recv(&temp, 1, MPI_DOUBLE, i, 1, MPI_COMM_WORLD, &status);
        total += temp;
    }
}
```

Part 3 - OpenMP Fundamentals and Programming

Exercise 2.9: OpenMP Parallel Loops (Coding Problem)

Parallelize the following serial matrix-vector multiplication using OpenMP:

```
#include <omp.h>
#include <stdlib.h>
#include <stdio.h>

void matrix_vector_mult(double** A, double* x, double* y, int m, int n) {
    // A is m x n matrix, x is n-vector, y is m-vector
    for(int i = 0; i < m; i++) {
        y[i] = 0.0;
        for(int j = 0; j < n; j++) {
            y[i] += A[i][j] * x[j];
        }
    }
}
```

Requirements: - Use `#pragma omp parallel for` with appropriate scheduling - Consider data dependencies and race conditions - Add timing with `omp_get_wtime()` - Test scalability with different thread counts

Exercise 2.10: OpenMP Data Scoping

Identify and fix scoping issues in this parallel region:

```
#pragma omp parallel for
for(int i = 0; i < N; i++) {
    int local_sum = 0;
    for(int j = 0; j < M; j++) {
        local_sum += data[i][j];
    }
    results[i] = local_sum;
    total += local_sum;  // Problem here!
}
```

Exercise 2.11: OpenMP Reductions

Implement parallel sum reduction with different approaches:

```
// Approach 1: Critical section (inefficient)
#pragma omp parallel for
for(int i = 0; i < N; i++) {
    #pragma omp critical
    total += array[i];
}

// Approach 2: Reduction clause (efficient)
// Your code here

// Approach 3: Manual reduction with partial sums
// Your code here
```

Part 4 - PThreads Fundamentals and Programming

Exercise 2.12: PThreads Producer-Consumer (Coding Problem)

Implement a producer-consumer pattern using PThreads with the following specifications:

- Buffer size: 10 integers
- 2 producer threads, 2 consumer threads
- Producers generate sequential integers
- Consumers print and consume integers
- Use mutex and condition variables for synchronization

Skeleton:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define BUFFER_SIZE 10
int buffer[BUFFER_SIZE];
int count = 0, in = 0, out = 0;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t not_empty = PTHREAD_COND_INITIALIZER;
pthread_cond_t not_full = PTHREAD_COND_INITIALIZER;

void* producer(void* arg) {
```

```
    // Producer implementation
}

void* consumer(void* arg) {
    // Consumer implementation
}

int main() {
    // Create producer and consumer threads
    // Wait for completion
}
```

Exercise 2.13: PThreads Synchronization Patterns

For each scenario, choose the appropriate PThreads synchronization primitive:

1. Multiple threads need to read a shared counter but only one can update it.
2. Threads must wait for all others to reach a synchronization point.
3. A thread pool needs to wait for any task to complete.
4. Initialize a shared resource exactly once, even with multiple threads.

Exercise 2.14: PThreads vs OpenMP Comparison

Compare and contrast PThreads and OpenMP for the following aspects:

1. Thread management
2. Data scoping
3. Synchronization
4. Performance considerations
5. Ease of use

Part 5 - Mini Design Studios (Realistic Scenarios)

Exercise 2.16: Matrix Multiply on a Cluster

Target: multiply C = A × B for square matrices of size N on a 16-node CPU cluster (each node: 2×16 cores, shared memory within node).

- Choose an overall memory paradigm and a programming model.
- Propose a data distribution (e.g., block, block-cyclic) and justify.
- Outline the communication steps you'd expect during the algorithm.
- Suggest how to exploit intra-node parallelism.
- Note one potential scalability bottleneck and one mitigation.

Exercise 2.17: 2D Heat Equation (Hybrid)

We solve a 2D heat diffusion with five-point stencil for T time steps on a 4-node cluster, each node has 32 cores.

- Propose a hybrid design (inter-node + intra-node) and explain why it fits Chapter 2 guidance.
- Sketch the halo exchange schedule.
- Describe how you'd tune granularity (domain tiles, OpenMP schedule, affinity/NUMA).

Exercise 2.18: Service Pipeline (Task Parallelism)

A media service performs: ingest → transcode → thumbnail → package → upload. Throughput is the KPI.

- Draw a 5-stage pipeline with queues between stages.
- Identify two backpressure points and mitigation strategies.
- Mention when you'd shard by content vs by stage.

Part 6 - Quick Checks (Selection Decisions)

Answer concisely (1–2 lines each), referencing Chapter 2 factors: problem, hardware, development constraints.

1. When is shared memory preferable to distributed memory?
2. When is data parallelism more natural than task parallelism?
3. Give one situation where a hybrid (MPI + OpenMP) is clearly justified.
4. Name a case where coarse granularity is preferred to fine granularity.
5. List two developer-centric reasons to avoid needless distribution.
